GymLog

An end-to-end project handbook — every technology, every test, every feature, and the reasoning behind all of them.

Written for: Adarsh Jain — so that on the next project you can ask the right questions instead of nodding along.

The app: a gym management and workout-tracking web application for one trainer and their members. Live at `gymlog-jtd8.onrender.com`. Source at `github.com/Adarsh-7codes/gym-log`.

Why this document exists. You said: *"I did not understand much, I was just nodding on your decisions and that is not the way of doing something with you."* That is the right instinct, and it is the single most useful thing you have said in this project. This handbook is written to close that gap. Every section answers three questions: **what is this**, **why did we choose it**, and **what would have happened if we chose differently**. The last two sections are the important ones — they tell you which of my decisions you should have pushed back on, and exactly what to ask me next time.

How to read this

- **Parts 1–2** build the mental model. Read these even if you skip everything else.
- **Part 3** is the architecture — the shape of the system. The most reusable section.
- **Parts 4–7** are the technical core: technologies, database, security, features.
- **Parts 8–9** are about testing — what we tested, what broke, and what I did about it. This is where most of the real learning is.
- **Parts 10–14** are honest self-assessment: weaknesses, what to improve, and the questions you should be asking.
- There is a **glossary** at the end. If a word is unfamiliar, it is probably there.

Part 1 — What we actually built

GymLog is a website that a gym trainer and their members both log into, on their phones. It does two jobs that pull in opposite directions, which is the whole design challenge of the project.

The two jobs

	The member's job	The trainer's job
Who they are	Comes to the gym 1–2 times a week. Standing between sets, sweaty hands, wants to be done in seconds.	Runs the business. Opens the app every day. Needs to know who owes money and who is about to quit.
What they need	**Speed.** Log a set with almost no typing.	**Scanning.** See 30 members at a glance and spot the 3 who need attention.
What kills it	Any friction. If logging takes 30 seconds, they stop using it.	Having to click into every member's profile to find out anything.

Those needs are opposite. A member wants a screen with almost nothing on it. A trainer wants a dense table with everything on it. That is why the app has **two different interfaces sharing one database** — not one interface with a few things hidden.

The single most important decision in the project

Partway through, the positioning changed: **the app is sold to the trainer, not the members.** The trainer is the only person guaranteed to open it daily. Members attend twice a week and, realistically, only 10–25% of them will ever type anything into an app.

That single sentence re-ranked every feature after it. "Does this help the trainer run or sell his business?" became the test. Member features survived only where they need **near-zero typing**. If you take one strategic lesson from this project, it is that **knowing who pays you determines what you build.**

What it does, in one paragraph

A trainer creates member accounts, assigns each member a weekly training split (Monday = chest and arms, and so on) and picks which exercises they should do. The member opens the app at the gym, sees only today's exercises, and logs weight and reps with plus/minus buttons. The trainer marks attendance with one tap per member, records membership payments and expiry dates, sets strength targets, and records weigh-ins. The app then tells the trainer who has stopped progressing, who has stopped showing up, whose membership expires this week, and who owes money.

Part 2 — The mental model: how a web app works

Skip this if it is familiar. If it is not, everything else will make more sense afterwards.

The six pieces

Piece	What it is	In GymLog
Browser	The program on the phone that shows pages and sends requests.	Chrome on your trainer's phone.
Request	A message: "give me this page" or "save this data".	`GET /dashboard`, `POST /logs/new`
Server	A computer, always on, that receives requests and decides what to do.	Our Python app, running on Render.
Application code	The rules. Who can see what, what happens when a button is clicked.	`app/routers/`, `app/crud.py`
Database	Permanent storage. Survives restarts.	PostgreSQL on Render.
Response	The finished page sent back to the phone.	The HTML the member sees.

What happens when a member taps "Save set"

1. The browser sends a **POST request** to `/logs/new` carrying the exercise, weight and reps.
2. The server checks the **cookie** that came with it to work out who is logged in. No valid cookie, no entry.
3. The application code checks **permission**: is this person allowed to write this record?
4. It writes a row to the `logs` **table** in the database.
5. It sends back a **redirect** telling the browser to load the log screen again.
6. The member sees "Saved" and the next set is ready. Total time: under a second.

The lesson to carry forward: step 3 is where security lives. It is tempting to think "the member can't see the delete button, so they can't delete it." That is false. Anyone can send any request by hand. **If the rule is not enforced in the code on the server, the rule does not exist.** We rebuilt this app's permissions around that fact in Phase 0.

Two things that confused you, cleared up

"Where is the database, and which one has the passwords?"

There are **two separate databases**, and they both contain *everything* — users, exercises, logs, memberships. They are not split by *type* of data. They are split by *location*:

- **Local (`gym.db` on your laptop)** — a SQLite file. Your test data. The live app never reads it.
- **Live (Render PostgreSQL)** — the real data your trainer creates. Your laptop can be switched off and it keeps running.

Think of two identical filing cabinets in two different offices: same drawers and labels, different paper inside. Neither can affect the other.

"Show me the members' passwords"

You cannot see them, and neither can I — **by design, not by limitation.** Passwords are stored *hashed* with bcrypt: scrambled one-way so they cannot be turned back. The database holds `$2b$12$hPRDJY2IXBF...`, not the password. When someone logs in, we hash what they typed and compare the scrambles.

This is correct and non-negotiable. If a database ever leaks, hashed passwords are useless to the attacker. Any app that *can* show you a user's password is storing them unsafely.

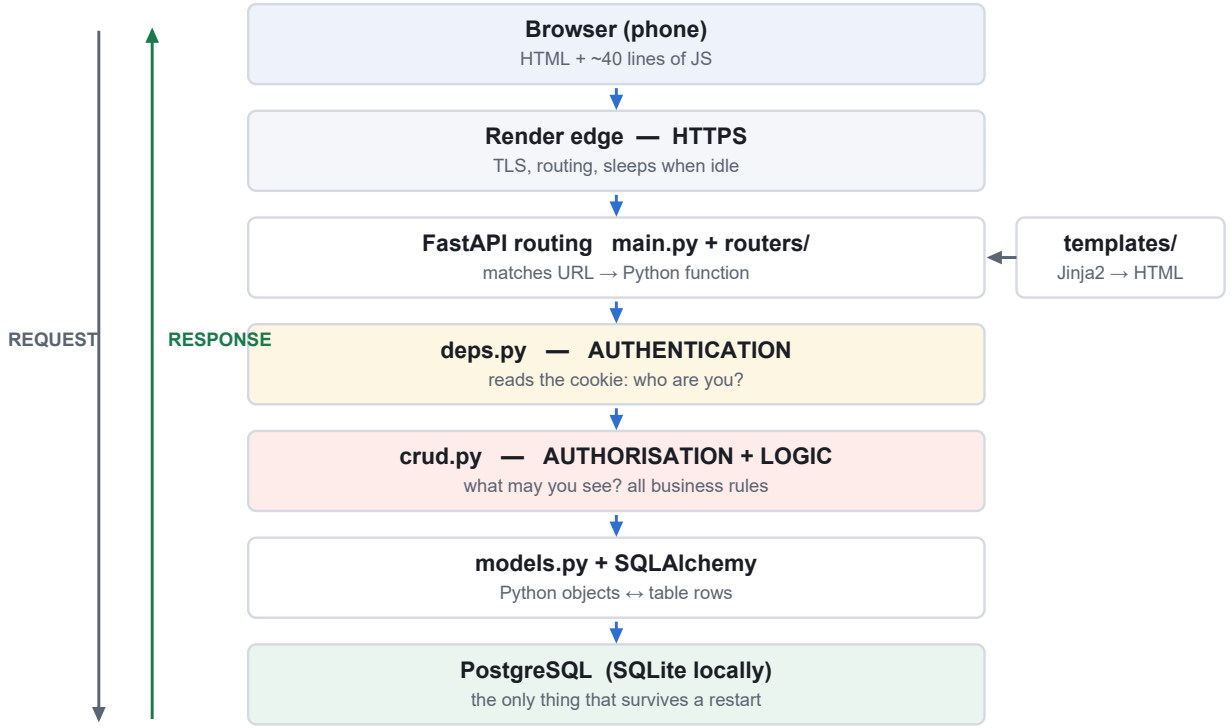
To *know* a member's password, the trainer sets it for them: Members → Add a member account → type a temporary password. You know it because you typed it.

Part 3 — The architecture

Architecture is the shape of the system — what the pieces are, which piece is allowed to talk to which, and where each kind of decision is made. It is not the code. You can read every line of this app and still not know that *authorisation lives in one file*. That fact is architecture, and it only exists if someone writes it down.

3.1 The layered view

Every request travels down through these layers and the answer travels back up. Each layer has exactly one job.



The two coloured layers are the ones that decide whether you are allowed to be here. Everything above them is plumbing; everything below is storage.

3.2 Trace one real request

A member taps **Save set**. Here is every layer it touches:

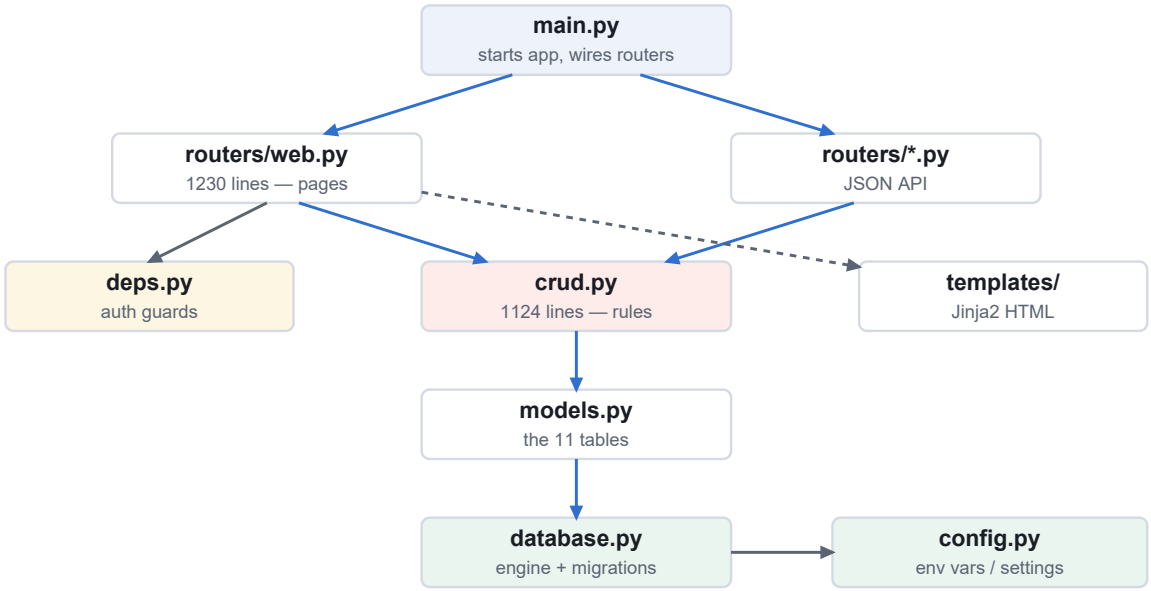
Layer	What it does	What it does NOT do
Browser	Sends `POST /logs/new` with exercise, weight, reps.	Decide anything. It can be lied to and bypassed entirely.
FastAPI route	Matches the URL, converts form fields to Python types.	Check permissions itself.
`deps.py`	Reads the cookie, validates the JWT signature, loads the `User`. No valid token → redirect to login.	Decide what that user may do.

Layer	What it does	What it does NOT do
`crud.create_log`	**Decides the owner.** A member can only write their own row; a trainer writing for a member records `logged_by=trainer`.	Trust the `user_id` the browser sent.
SQLAlchemy	Turns the object into a safe, parameterised `INSERT`.	Allow raw text into SQL (this is what blocks injection).
Database	Stores the row permanently.	Enforce business rules.

Read the fourth row again. That is the entire security model of this application in one sentence: *the layer that touches the data decides who owns it, and it never trusts what the browser said.* If you understand only one thing about this architecture, make it that one.

3.3 The module map

Which file does what, and — more importantly — which way the arrows point. Arrows mean *depends on*.



Solid = calls into. Dashed = renders with. Notice there are **no arrows pointing back up** — `crud.py` knows nothing about routes, and `models.py` knows nothing about `crud.py`.

Why one-way arrows matter so much

A system where A depends on B, and B also depends on A, cannot be understood, tested or changed one piece at a time. Every change ripples both ways. This is how projects become the thing people call a *big ball of mud*.

Because our arrows only point downward, you can read `crud.py` on its own and fully understand it. You can test it without starting a web server. And you could replace the entire web layer with a mobile API tomorrow without touching a single business rule. **That property is worth more than any individual feature in this app.**

3.4 The rules this architecture runs on

Rule	Why	Enforced how?
Authorisation lives in `crud.py`, never in a template.	Hiding a button is not security. Anyone can send the request by hand.	By convention + the authorisation tests. **Not by the compiler.**
Routes never trust a `user_id` from the browser.	This is the IDOR attack. `resolve_target_user()` silently forces a member back to their own id.	One shared function, used everywhere.
Trainer-only routes use the shared `require_trainer_web` dependency.	A copy-pasted check is a check someone will forget.	One dependency, attached per route.
Schema changes extend, never rewrite.	The live database holds real data. Adding is safe; altering is not.	`ensure_schema()` on every startup, idempotent.
The same code runs on SQLite and PostgreSQL.	So local development and production cannot silently diverge.	SQLAlchemy; no dialect-specific SQL anywhere.

An honest weakness, found while writing this section. I measured it rather than assumed it: `web.py` makes **73 calls into `crud.py` but also 27 database queries directly.** I checked all 27 — every one is harmless reference data (lists for dropdowns, a lookup by id) and **no permission-sensitive query bypasses `crud.py`**. So the app is safe today.

But the *rule* exists only in my head and in this paragraph. Nothing in the code stops the next person adding a permission-sensitive query straight into a route and quietly stepping around the entire security layer. **That is exactly the kind of erosion an architecture document is supposed to prevent** — which is a fitting thing to discover while writing one.

3.5 The data model

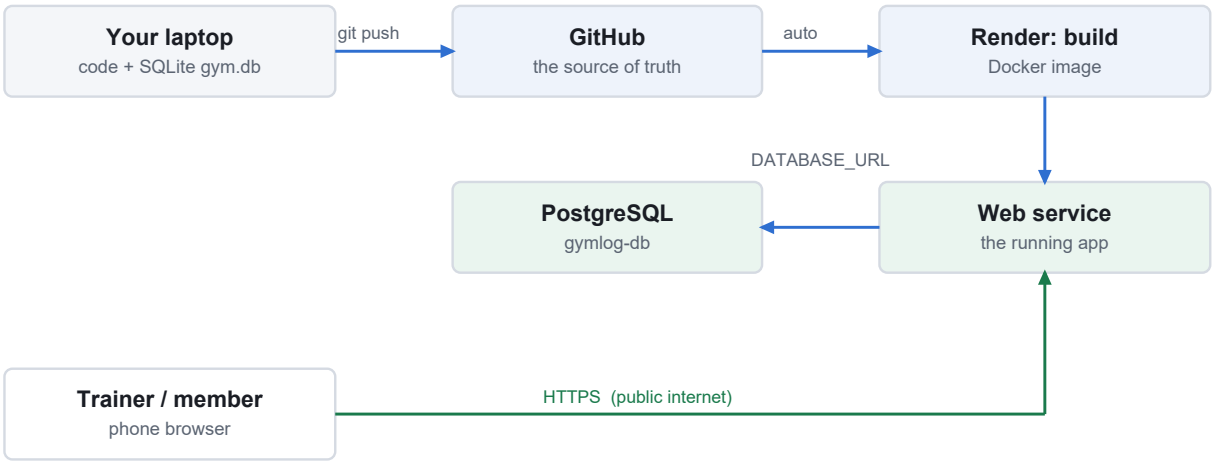
Eleven tables. `users` and `exercises` are the two anchors; almost everything else hangs off one or both.

`logs`, `member_routines` and `targets` link a **member** to an **exercise**. `attendance`, `memberships` and `body_weights` belong to the member alone.

The one relationship that matters most

`member_routines` (what you plan to do) and `logs` (what you actually did) are **separate tables on purpose**. Delete an exercise from your routine and it stops appearing tomorrow, but every session you ever did survives. If those were one table, changing your mind would destroy your history. *There is a test whose only job is to keep that true.*

3.6 The deployment architecture



One `git push` is the entire deployment. Render rebuilds the Docker image and swaps it in. **Your laptop's SQLite file is never involved** — it cannot be, which is the point.

Why this diagram is worth having. When something goes wrong, this tells you *where to look*. Your email-validation bug is the perfect example: the code on GitHub was correct, but the phone was hitting a server

process that had never restarted. Without this picture you debug the code — which was fine. With it, you ask *"which box is actually running my change?"* and find it in seconds.

Why the architecture document is the most important one

You asked why this document matters more than the others. Here is the honest answer, and it is worth internalising because it applies to every project you will ever work on.

1. Code tells you *what*. Only architecture tells you *where*.

Every fact in this project can be recovered by reading 3,265 lines of Python — eventually. But the questions you actually need answered are locational: *Where do I add a new feature? Where is permission decided? What will break if I change this?* Code answers none of those quickly. **Architecture is the index to the codebase.**

2. It is the only way to answer security questions at all

"Prove a member cannot read another member's payment history." Without an architecture you must inspect every route, forever, and you can never be sure you found them all. With one, the answer is: *all member data access passes through* `crud.py`*, here is that file, here are the tests*. **You cannot audit a system you cannot map.**

This is not theoretical for GymLog. It stores names, emails, attendance and payment records. The moment a real gym uses it, someone may reasonably ask that question.

3. It defines the blast radius of a change

The single most expensive question in software is *"what else did that break?"* A dependency map answers it directly: change `models.py` and everything below is affected; change a template and nothing is. Without the map, every change is a gamble, so people either move slowly or break things. **Both are expensive.**

4. It survives people; memory does not

Right now the reasoning behind this app lives in two places: this document, and a conversation that will eventually be forgotten. In six months *you* will be the new developer on this project. Architecture is a letter to that person.

> *The people who made the decisions leave. The decisions stay. Only the written ones can be revisited on purpose — the rest get discovered by accident, usually at the worst time.*

5. Without it, layers blur — and that is irreversible

This is the failure mode to genuinely fear. Nobody destroys an architecture deliberately. It goes like this: someone is in a hurry and puts a database query in a template. It works. The next person copies the pattern. Two years later every file talks to every other file, nobody can change anything safely, and the only remaining options are a rewrite or abandonment.

The 27 direct queries I found in `web.py` **are exactly this process, at step one.** They are harmless today. They are harmless *because I checked*, not because anything prevents them. Writing the rule down is the cheapest possible intervention; a rewrite in two years is the most expensive.

6. It is what a buyer, auditor or investor asks for

If you sell this to a gym chain, license it, or put it in a portfolio, the competent question is never "how many features does it have?" It is *"how is it structured, where is the data, and who can reach it?"* Having a real answer separates a product from a demo.

7. Writing it finds bugs — demonstrably

I did not intend to find anything writing this section. Measuring the layering to draw an accurate diagram is what surfaced the 73-versus-27 split. **The act of explaining a system forces you to look at what is actually there rather than what you remember building.** That is the same reason explaining a problem out loud often solves it.

The test of a good architecture document: hand it to a competent developer who has never seen the project. If, after twenty minutes, they can correctly say *"to add a 'trainer notes' feature I'd add a table in* `models.py`*, the rules in* `crud.py`*, a route in* `web.py`*, and a panel in* `dashboard.html` *— and I'd need an authorisation test"* — then the document works. If they have to read the code first, it does not.

What to do with this on your next project

- **Draw it before you build it**, even roughly. Boxes and arrows on paper. It takes ten minutes and it is where you notice that two things you were treating as separate are actually the same thing.
- **Write down the rules, not just the boxes.** "Authorisation lives in X" is worth more than any diagram.
- **Update it when you break a rule** — deliberately or not. An architecture document that describes an app that no longer exists is worse than none, because it is trusted.
- **Ask me for it.** On your next project, *"draw me the architecture and tell me which rules it depends on"* is one of the highest-value things you can ask, and you should ask it *early* — not, as here, at the end.

Part 4 — Every technology, and why

For each one: what it is, why we picked it, what we *could* have picked, and the honest trade-off we accepted.

Python

What: the programming language the whole back end is written in.

Why: it reads almost like English, has the largest library ecosystem for this kind of work, and is the language most commonly taught — so you can actually maintain this yourself.

Alternatives: Node.js (JavaScript everywhere), Go (much faster, much more verbose), PHP (still runs most of the web).

Trade-off accepted: Python is comparatively slow. For a gym with a few hundred members that is completely irrelevant — the database and the network are the bottleneck, not the language. *Choosing a fast language for a small app is optimising the wrong thing.*

FastAPI

What: the web framework. It turns a URL like `/dashboard` into a Python function.

Why: it is modern, fast, and gives you **automatic interactive API documentation** at `/docs` for free. It also validates incoming data automatically — if a form should have a number and gets "abc", FastAPI rejects it before your code runs.

Alternatives: Django (much bigger, comes with an admin panel and user system built in), Flask (smaller, more manual).

Trade-off: Django would have given us a ready-made admin panel and login system — genuinely less work up front. We chose FastAPI for its lighter footprint and speed. **Honestly: for this project Django would have been a defensible, possibly better choice.** This is a decision worth questioning me on.

SQLAlchemy

What: the ORM — *Object Relational Mapper*. It lets us write `db.get(User, 5)` in Python instead of `SELECT * FROM users WHERE id = 5` in SQL.

Why: two big wins. First, it is **the reason the same code runs on SQLite locally and PostgreSQL in production** — SQLAlchemy speaks both dialects. Second, it protects against **SQL injection**, the classic attack where someone types SQL into a form field and takes over your database.

Trade-off: a layer of abstraction to learn, and it can generate inefficient queries if you are careless (see the N+1 problem in Part 9).

Jinja2 templates + server-rendered HTML

What: HTML files with placeholders. The server fills in the data and sends finished HTML.

Why: this is the decision that keeps the app **fast on a bad gym wifi connection**. The phone receives a finished page and displays it. There is no JavaScript framework to download, parse and boot up first.

Alternatives: React, Vue, Angular — where the browser downloads an application and then fetches data separately.

Trade-off: every action reloads the page. A React app would feel smoother and more 'app-like'. For a form-heavy tool used in short bursts, server rendering is simpler, faster to load, and about a third of the code.

The brief explicitly forbade adding a JS framework, and I agree with that call.

Vanilla JavaScript (a small amount)

What: plain browser JavaScript, no library, used in three places only: the plus/minus steppers, revealing the set-entry fields after picking an exercise, and the exercise search filter.

Why: these need instant feedback, and instant feedback requires code running in the browser. About 40 lines total — not worth a framework.

bcrypt (password hashing)

What: the algorithm that scrambles passwords one-way.

Why bcrypt specifically: it is **deliberately slow**. That sounds like a flaw and is actually the point. An attacker who steals the database has to try billions of guesses; if each guess takes 0.1 seconds instead of 0.000001, cracking becomes impractical. It also automatically adds a random *salt* to each password, so two members with the same password get different hashes.

Never do instead: MD5 or SHA-1. They are fast, which is exactly wrong for passwords.

JWT (JSON Web Tokens) in an httponly cookie

What: after login, the server gives the browser a signed token proving who you are. The browser sends it with every request.

Why: the server does not have to remember who is logged in — the token itself carries the proof, signed with a secret only the server knows. Tamper with it and the signature breaks.

Why `httponly`: the cookie is marked so **JavaScript cannot read it**. If someone ever injected a malicious script into a page, it still could not steal the login token.

Why `secure=true` in production: the cookie is only ever sent over HTTPS, so it cannot be read off public wifi.

Docker

What: packages the app *and* its exact Python version and libraries into one image that runs identically anywhere.

Why: it kills "it works on my machine". Render builds the same image we would build locally.

Inline SVG for all charts

What: the progress charts are drawn as SVG shapes generated by our own Python code.

Why: a charting library like Chart.js would add roughly 200KB for the member to download on gym wifi, plus a dependency to keep updated and patch for security. Our charts are lines and rectangles — roughly 60 lines of maths. **Zero dependencies, zero download, works with JavaScript disabled.**

Trade-off: no interactive tooltips or zoom. We get hover labels via the SVG `<title>` element, which is enough here.

email-validator

What: properly validates email addresses.

Why it exists in this project: you found the bug. `jainandadarsh7423@g` was being accepted. The browser's built-in `type="email"` check is deliberately loose — it only wants an `@`. This library checks the domain is structurally real. *Browser validation is a convenience for the user, never a guarantee for the server.*

The stack in one table

Layer	Choice	One-line reason
Language	Python 3.12	Readable, huge ecosystem, you can maintain it
Web framework	FastAPI	Fast, auto-validates input, free API docs
Database access	SQLAlchemy 2.x	One codebase runs on SQLite and PostgreSQL; blocks SQL injection
Templates	Jinja2	Server-rendered = fast on bad wifi
Front end	HTML/CSS + ~40 lines JS	No framework to download; brief forbade one
Charts	Hand-generated inline SVG	No dependency, no download weight
Passwords	bcrypt	Deliberately slow + salted
Sessions	JWT in httponly cookie	Stateless, unreadable by JavaScript
Local database	SQLite	Zero setup, it is just a file
Live database	PostgreSQL	Real concurrency and durability
Packaging	Docker	Identical everywhere
Hosting	Render (free tier)	Free, auto-deploys from GitHub, managed database
Version control	Git + GitHub	History, and it is what triggers deployment

Part 5 — The database, explained properly

Why two different database engines?

This is the question you asked, and it is a good one. We use **SQLite** on your laptop and **PostgreSQL** on the live server. Same data structure, deliberately different engines.

	SQLite (local)	PostgreSQL (live)
What it is	A single file, `gym.db`. No server process.	A real database server.
Setup	None. It just works.	Managed by Render.
Concurrency	One writer at a time.	Many writers at once.
If the power cuts	Usually fine, occasionally corrupt.	Built for durability.
Why we use it	Instant setup, delete the file to start over.	Trainer + members hitting it at once, safely.

The principle: use the simplest thing that works in development, and the robust thing in production — but make sure *the same code runs on both*. That is exactly what SQLAlchemy buys us. It is also why we never wrote database-specific SQL: it would have worked locally and broken live.

The gotcha that would have broken the deploy

Render hands out its database address starting with `postgres://`. SQLAlchemy 2.x only accepts `postgresql://`. One missing three letters and the live app would refuse to start.

```
if v.startswith("postgres://"):
    v = "postgresql://" + v[len("postgres://"):]
```

Six lines in `app/config.py`, added *before* deploying rather than debugging at midnight. **This is what "knowing the platform" means in practice.**

The tables, and what each one is for

Table	Holds	Why it exists separately
`users`	Name, email, hashed password, role (trainer/member)	One row per person. The role column is the entire permission system.
`exercises`	48 seeded exercises with body part, difficulty, equipment, instructions	A shared library. Not hardcoded in the page, so it can be edited without a code change.
`member_routines`	Which exercises a member does, per body part, and who assigned it	A member's *standing* selection. Deleting a row stops it appearing in future — it does **not** touch past logs.
`split_days`	Which body part(s) on which weekday	Several rows per day allowed, so "Monday = chest + arms" works.
`logs`	Every set: date, weight, reps, sets, how it felt, who entered it	The permanent performance history. Never deleted by routine changes.
`attendance`	One row per member per day they showed up	Ground truth for "did they come", independent of whether they logged anything.

Table	Holds	Why it exists separately
`memberships`	Plan start, duration, expiry, amount, paid/pending	Several rows per member = renewal history. This is the revenue record.
`targets`	Trainer-set goal: exercise, weight, reps, by date	Checkable against `logs`, which is what makes it honest.
`body_weights`	One weigh-in per member per day	Trend only. Deliberately no goal percentage.

Three schema decisions worth understanding

1. Routine and history are separate tables — on purpose

If a member drops "Bench Press" from their routine, their bench press history must survive. If those lived in one table, removing the exercise would delete months of progress data. Two tables, and a deliberate rule: **removing from `member_routines` never touches `logs`.** We wrote a test specifically to prove this stays true.

2. `expires_on` is calculated once and stored

We could calculate a membership's expiry every time we display it. Instead we compute it when saving and store it. **Why:** the roster sorts and filters by expiry across every member on every page load. Calculating on read would mean doing that maths hundreds of times per page. *Store what you sort by.*

3. "Extend, never rebuild"

Every schema change added a column or a table. We never altered or dropped an existing one. **Why:** the live database has real data in it. A rebuild means data loss or a risky migration. Adding is safe; changing is not.

New columns are added by `ensure_schema()` in `app/database.py`, which runs on every startup, checks whether the column already exists, and adds it if not. Safe to run a thousand times.

Part 6 — Security, and why it came before the money features

The rule we followed: *Phase 1 adds financial records to the database. Do not add them to an app with broken authorisation.* Security work is boring and invisible until the day it isn't. Doing it *before* storing payment data, rather than after, is the difference between a fix and an incident.

What we actually did in Phase 0

Problem	Fix
Permission checks were copy-pasted inline in each route — easy to forget one.	A single shared `require_trainer_web` dependency. Attach it to a route and the check is guaranteed.
Anyone who found the URL could register an account.	Self-registration closed once a trainer exists. The trainer creates members. One bootstrap exception so an empty database can create its first (trainer) account.
A `/danger/reset` URL existed on the live site that could wipe the database.	Made **local-only**: it now requires both SQLite *and* a token. On the live PostgreSQL server it returns 404 permanently, whatever the environment variables say.
No proof that any of it worked.	`docs/authz-check.md` — a re-runnable list of curl commands that attempt every attack and record the expected 403/404.

The three layers of defence

1. **The UI** hides what you cannot use. This is *courtesy, not security* — it stops confusion, not attackers.
2. **The route** checks your role before running (`require_trainer_web` returns 403).
3. **The data layer** (`crud.py`) filters by owner regardless of what was asked for. A member requesting `?user_id=7` silently gets their own data. Another member's record returns **404, not 403** — deliberately, because 403 would confirm the record exists.

The vulnerability class this prevents: IDOR

Insecure Direct Object Reference — the most common serious flaw in apps like this. You are member #4, you notice the URL says `?user_id=4`, you change it to `?user_id=5`, and you are reading someone else's payment history. It has caused real breaches at real companies.

We tested this explicitly, repeatedly, in every phase. The member either gets their own data, a 403, or a 404 — never someone else's.

Secrets: what is and is not in the repository

- `.gitignore` excludes `.env` and `*.db`, so credentials and databases are never committed.
- `JWT_SECRET` is **generated by Render** at deploy time. Nobody typed it; it exists only in Render's settings.
- The database password lives only in Render's `DATABASE_URL`, injected at runtime.
- The one secret-looking string in the code, `"dev-secret-change-me"`, is a placeholder that is overridden in production.

- The reset script prints only the *host* part of a database URL, so a password cannot leak into terminal scrollback.

Part 7 — Every feature, both perspectives

For each feature: what the member sees, what the trainer sees, and — the part that matters — **why it earns its place.**

1. Exercise Library (48 exercises)

- **Member:** browses by body part (chest, back, legs, shoulders, arms, core), sorted beginner → advanced. Ticks the ones they want; they stay in their routine permanently.
- **Trainer:** can add/edit exercises, including an optional demo video URL.
- **Why it matters:** a beginner does not know what to do. Sorting by difficulty means they start at push-ups, not barbell bench press. **Business impact:** beginners who feel lost quit; this is a retention feature disguised as a list.

Design note: exercises come from the database, never hardcoded in the page. The trainer can change the library without a developer.

2. Weekly Split

- **Member:** assigns body parts to weekdays — Monday = chest + arms. Multiple body parts per day supported, because people really do train chest and triceps together.
- **Trainer:** can set any member's split for them.
- **Why it matters:** this is the feature *you* asked for, and it was the right call. It turns a 48-item dropdown into a 4-item list. On Monday you see Monday's exercises. That is the difference between an app someone uses at the gym and one they abandon.

3. Fast logging (the member's core screen)

- **Member:** taps an exercise from today's list → weight and reps are **pre-filled with last session's numbers** → adjusts with +/− buttons → taps "How did it feel?" → one big Save button. Stays on the same screen for the next set.
- **Trainer:** has a fuller form and can log on a member's behalf (recorded as `logged_by = trainer`).
- **Why it matters:** count the taps. Old way: scroll a 48-item dropdown, type 60, type 8, type 3. New way: tap, tap, save. **The auto-carry of last session's weight is the single highest-value detail in the app** — most sessions repeat the previous weight, so the common case requires zero typing.

Technical detail with real-world impact: the number fields use `inputmode="decimal"` and `inputmode="numeric"`, so phones open the *number pad* instead of the full keyboard. And all inputs are 16px, because iOS Safari zooms the whole page in on any smaller field.

4. Attendance (Phase 3)

- **Trainer:** a "Today" screen — one large tap target per member, designed for one-handed use at the desk. Tap again to undo. Unmarked members sort to the top so the remaining list shrinks as you work.
- **Member:** sees their own sessions this month and a week streak. Read-only, no input.
- **Why it matters:** this fixed a genuine flaw. Before it, "inactive" was measured from workout *logs*. A member who trained six times but never logged looked identical to one who quit. The flag was worse than useless — it was misleading. **Any reminder system built on that data would have nagged loyal members.**

5. Membership and dues (Phase 1)

- **Trainer:** each member's plan, expiry date, payment status and full renewal history. One-click "Mark paid". The roster shows **Expires** and **Dues** columns with a summary strip: active / expiring within 7 days / expired / total pending rupees.
- **Member:** "Your membership is valid till 18 Nov 2026" plus payment history. Read-only, and deliberately **no dues-chasing language**.
- **Why it matters:** this is the feature that makes the app a purchase rather than a toy. The gym owner's daily question is *"who owes me money and whose membership expires this week?"* Before this, that lived in his head or a notebook.

Deliberately excluded: payment processing, UPI integration, card storage, GST invoicing. Those bring legal and PCI-compliance obligations far beyond this project. The trainer collects cash or UPI himself and records the outcome.

6. Stall detection

- **Trainer:** members whose lifts have plateaued are flagged automatically and sorted to the top of the roster.
- **How it works:** for each exercise, compare the last 3 sessions' best set as `(weight, reps)`. If the newest does not beat the best of the previous two, it is stalled.
- **Why it matters:** a trainer with 30 members cannot remember everyone's bench press. This is the app doing the remembering. **Business impact:** a member who stops progressing quietly quits; this is the early-warning system.

7. Talking points and the first-90-days view (Phase 4)

- **Trainer:** up to three short factual lines on each member's page — *"Barbell Bench Press 40 → 47.5 kg over 6 weeks"*, *"12 sessions this month, up from 8"*, *"Hasn't trained legs in 18 days"*. Plus a cohort tab for members who joined in the last 90 days, where most churn happens.
- **Member:** sees none of it. These are notes for the trainer to speak from, not a verdict handed to the member.
- **Why it matters:** value reaches the member through the trainer's mouth, not a chart. On a floor with 30 people, this gives him something specific and true to say to each one.

The rules encoded in this feature, and why they are in the code rather than left to the interface: facts only, drawn from logged or attendance data; **never** infer *why* something happened (diet, effort, motivation, discipline — the app cannot observe these); never use accusatory phrasing; and **if there is not enough data for a true statement, show nothing rather than filler.** A member with two logs produces zero lines and the page honestly says "Not enough data yet". We wrote a test that fails if a judgemental word ever appears.

8. Progressive-overload targets (Phase 5)

- **Trainer:** sets a target — exercise, weight, optional reps, by a date.
- **Both see the gap:** *"60 kg by 15 Oct · Current best 45 kg · 15 kg to go · Unchanged for 3 weeks"*, with reached and overdue states.
- **Why it matters:** it is completely objective. Every number is checkable against the logs, which is exactly why it is worth showing the member.

Subtle detail: "unchanged for 3 weeks" is measured from when the current best was *first* reached, not from the last session. That measures the actual plateau.

9. Body weight (Phase 6)

- **Trainer:** records weigh-ins, weekly cadence expected.
- **Both see:** *"Down 2.1 kg over 6 weeks — about 0.35 kg/week"* with a trend line smoothed over a 7-day rolling average.
- **Why the smoothing:** body weight swings 1–2 kg on water alone. Without smoothing, a normal fluctuation looks like failure.

What this feature deliberately does NOT have, and why this is the most ethically important decision in the app:

No progress bar or "% of goal complete." Body weight is not monotonic — it goes up and down week to week for reasons nobody controls. A bar that moves backwards punishes a member who did everything right.

No hard-coded target rate. A goal like "10 kg in 2 months" is about 1.25 kg/week, well above what is generally

considered sustainable. Encoding that into the UI would push every member toward an unhealthy expectation.

No attribution to diet, effort or discipline. The app cannot observe what someone eats or how hard they tried, so it must never claim to. It reports numbers; the trainer provides the judgement.

These prohibitions are **asserted in the test suite**, so they cannot quietly creep back in later.

10. Role separation, made visible

- First account ever registered becomes the **trainer**; everyone after is a **member**.
- Login has a Trainer | Member toggle that must match the account, with a helpful error if not.
- Trainer = gold theme, member = blue. Badge under the name, and a "Trainer view / Member view" banner.
- **Why it matters:** you were confused about which role you were in — and if *you* were, a 45-year-old trainer certainly would be. Visible role state is a usability feature, not decoration.

Part 8 — Testing: what, why, and what it caught

First: what a test actually is

A test is a small program that uses your app the way a user would, then **asserts** that something specific is true. If the assertion is false, it prints FAIL.

```
c.post("/logs/new", data={...})           # do the thing
logs = c.get("/dashboard")               # look at the result
assert "102" in logs.text                 # is it what we expected?
```

That is the whole idea. The value is not in any single test — it is that you can re-run **all** of them in seconds after a change, and instantly know whether you broke something you were not thinking about.

The four kinds of test we wrote, and why each

Kind	What it checks	Why we needed it here
Smoke test	Does the app start and serve a page at all?	Catches catastrophic breakage in one second before wasting time on detail.
Feature / acceptance test	Does the thing the brief asked for actually work?	Each phase had written acceptance criteria; these prove they are met.
Authorisation test	Can a member reach data or actions they should not?	The highest-stakes category. Money and personal data are involved.
Regression test	Does everything built earlier still work?	The most valuable kind by far. Six phases of changes to shared files — without these we would have been breaking Phase 1 while writing Phase 5 and not known.

The lesson: the regression tests earned their keep more than anything else. Every phase touched `crud.py`, `web.py` and `dashboard.html` — files the previous phases depended on. Re-running the old checks after each phase is what made it safe to keep building at speed.

What we tested, phase by phase

Suite	Checks	Examples of what it asserts
Original app smoke (API)	11	Register, login, roles, 401 without a token, member cannot read another's logs
Original app smoke (web)	11	Cookie login, redirect when logged out, member sees only own data
Edit / filters / members / charts	15	Edit persists, exercise and date filters, chart renders, member blocked from trainer pages
Weekly split	9	Multi-body-part days, day filtering, rest days, logs survive split edits
Library + routine + stall detection	16	`logged_by` recorded, stall flags, roster ordering, routine deletion keeps logs
Email validation	7	Rejects `ada@12`, `nope`, `mo@bad`; accepts real addresses
Login role toggle	8	Right tab works, wrong tab blocked with a helpful message

Suite	Checks	Examples of what it asserts
Demo reset endpoint	8	Disabled without token, 403 on wrong token, library preserved
Phase 1 — membership & dues	16	Expiry maths, red/amber states, sorting, dues total drops on payment, member blocked from writes
Phase 2 — trainer edits routines	14 + 5	Trainer assigns 4 exercises, member sees exactly those, forged `user_id` blocked
Phase 3 — attendance	16 + 6	Double-tap makes one row, un-marking, attendance drives inactivity not logs
Phase 4 — talking points	14 + 8	Correct improvement line, **zero** lines when data is thin, no judgemental words
Phase 5 — targets	16 + 10	Gap maths, plateau duration, reached/overdue, member read-only
Phase 6 — body weight	16 + 15	Trend and rate, no progress bar, no diet language, no hardcoded rate
Migration safety	2	Old database gains new columns and backfills correctly
Reset script	4	Wipes accounts, keeps the 48 exercises, abort path safe

Roughly 220 individual assertions in total, of which about 136 belong to Phases 1–6. Every suite ended green before its phase was committed.

Part 9 — What failed, and what I did about it

This is the section you specifically asked for, and the most useful one. **A test that never fails is not testing anything.** Here is every failure, honestly categorised.

The critical distinction: a red FAIL does not always mean the *app* is broken. Sometimes the *test* is wrong. Confusing the two leads to "fixing" working code until it breaks. Every time something failed here, the first question was: *is the app wrong, or is my test wrong?* The table below is honest about which it turned out to be.

Every failure, in order

#	What failed	Real bug or test error?	What I did
1	First smoke test: `no such table: users`	**Test error**	The test client was not started as a context manager, so the app's startup code (which creates tables) never ran. Fixed to `with TestClient(app)`.
2	Dashboard filter: "exercise filter FAIL"	**False alarm**	My check searched the whole page for "Back Squat", which also appears in the filter dropdown. Re-verified by counting actual table rows: 4 total → 1 bench, 3 squat. Correct all along.
3	Weekly split: 4 checks failed, split saved as empty	**Test error**	I sent the form data as a list of tuples, which the HTTP client did not encode as repeated checkbox fields. Real browsers do. Fixed the test encoding; app was correct.
4	Migration test: `no such column: assigned_by`	**REAL BUG**	`ensure_schema()` returned early if the `logs` table was missing, silently skipping every later migration. Harmless today, would have bitten us later. **Fixed the app** so each migration guards its own table.
5	Phase 4 test crashed with `UnicodeEncodeError`	**Environment**	The Windows console cannot print the arrow character. Nothing to do with the app, which outputs UTF-8 HTML. Re-ran with `PYTHONIOENCODING=utf-8`.
6	Phase 5 test crashed: `DetachedInstanceError`	**Test error**	I read a database object after closing its session. Captured the values first, re-ran.
7	Phase 6: "Down 2.1 kg" not found	**Test error + minor real flaw**	My test changed the data mid-run (2.1 → 2.5 kg) then asserted the old number. But it also revealed the page said "6.0 weeks" instead of "6 weeks" — **a real cosmetic flaw I fixed.**
8	A test file named `re.py`	**Test error**	It shadowed Python's built-in `re` module and broke the interpreter. Renamed.

Bugs caught by reading the code, not by tests

Worth recording, because it shows tests are not the only safety net:

What	How it was caught	Why it mattered
Targets panel nested inside the trainer-only block	Checked the template's block structure before trusting it	**Members would not have seen their own targets** — the brief explicitly requires both sides see the gap.
CSS class `.exrow` used for two different things	Grepped before adding the style	The new quick-log row styling would have silently broken the library's checkbox list.
A form field named `status` shadowing FastAPI's `status` module	Noticed while writing the route	Would have crashed on every membership save.
Live app still accepting `...@g` after the fix	You reported it; I checked the running server	**The code was already correct** — the phone was hitting an old server process that had not reloaded. Restarted it. *Always verify which version is actually running.*

The honest scorecard

- **8 test failures** across the project. **1 was a genuine application bug** (the migration guard). **1 revealed a genuine cosmetic flaw** ("6.0 weeks"). **5 were my own test-writing mistakes. 1 was a Windows console limitation.**
- **4 further bugs were caught by reading code** before they ever ran — including one (the targets panel) that would have broken a stated requirement.
- **Every phase's suite was green before that phase was committed.** Nothing was committed on a red test.

What this tells you about testing, and it is not the flattering version: most test failures in practice are the *test* being wrong, not the code. That is normal and fine. The discipline is investigating *which* it is every single time, rather than assuming. The one time it was a real bug (#4), it was a bug no human would have found by clicking around, because it only appeared in a specific database state.

Part 10 — What is weak, honestly

Every project has debt. Naming it is how you stay in control of it. Ordered by how much it would matter.

1. The tests are not in the repository — the biggest weakness

Every test in this project was written as a **throwaway script**, run, and deleted. There is no `tests/` folder, no `pytest`, no automation.

Why that is bad: nobody can re-run them. If you change one line next month, you have no way to check you did not break Phase 3. All that verification work is gone — only this document records that it happened.

The fix: move them into a `tests/` directory, run with `pytest`, and add a GitHub Action so they run automatically on every push. Perhaps a day of work. **If you do one thing to this project, do this.**

2. No automated deployment checks (CI/CD)

Right now `git push` deploys straight to the live site. If the code were broken, it would deploy broken. A CI pipeline would run the tests first and refuse to deploy on failure.

3. Performance: the N+1 query problem

The roster loads each member, then runs several more queries *per member* for attendance, membership and stall detection. With 20 members that is roughly 100 queries per page load. Fine now, visibly slow at 500 members.

The fix: batch them into grouped queries (we already do this for two of them). Worth doing when it hurts, not before. *Premature optimisation is its own bug.*

4. No password reset

If a member forgets their password, there is no self-service recovery. The trainer must create a new account.
The fix: a trainer-facing "reset this member's password" button — genuinely small, and the most likely thing to annoy a real user first.

5. Free-tier hosting limits

- The app sleeps after ~15 minutes idle; the next visit takes 30–60 seconds to wake.
- Render's free PostgreSQL **expires after about 30 days**. This is the one that can actually lose data — before any real use, upgrade or export.

6. No backups

There is no automatic export of the live database. For a demo that is acceptable. The day a real trainer records real payments, it is not.

7. Smaller items

- The old "Planner" feature overlaps with Weekly Split and should be retired.
- Weights display as `100.0` in some places, `100` in others.
- No pagination — a member with 2,000 logs loads all of them at once.
- Only one trainer is supported by design. A second coach needs a database edit.

- No audit trail of who changed what and when (we record *who logged*, but not edits).

Part 11 — If this becomes a real product

What changes when it stops being a demo and someone depends on it. In priority order.

Before a single real gym uses it

1. **Paid database with backups.** Non-negotiable. The free tier expires and takes the data with it. This is a business-ending risk, not a technical one.
2. **Commit the test suite and add CI.** Everything above depends on being able to change the app without fear.
3. **Password reset flow.** The first support request you will ever get.
4. **A written data-protection position.** You are storing names, emails and payment records. Know what you would say if a member asked you to delete their data.

The next features that would actually earn money

Feature	Why it is worth building	Why we did NOT build it yet
WhatsApp / SMS reminders	The highest-value feature on the list in India. "Your membership expires in 3 days" recovers renewals automatically.	It depends on attendance data being trusted. Built on the old broken inactivity signal it would have nagged loyal members. Now that Phase 3 exists, this is the right next step.
Multiple trainers	Any gym with more than one coach cannot use this today.	Deliberate simplification for a single-trainer product. It is a real ceiling on who can buy it.
Per-set logging	Right now one row is one session's top set. Serious lifters want set 1, set 2, set 3 separately.	Correct long-term, but it is a member-side feature and this is a trainer-first product.
Progress photos	Visually powerful for retention and for selling.	Needs file storage, and photos of people carry real privacy weight. Not to be added casually.
Exportable reports	A monthly PDF of revenue and attendance is something an owner would pay for on its own.	Not asked for yet. Cheap to add now that the data model exists.

What I would do differently from the start

- **Write the tests as a committed suite from day one.** Not throwaway scripts. This is the clearest mistake in the project, and it was mine.
- **Settle the positioning before building.** We built member-first, then re-positioned to trainer-first. Everything survived, but the Planner feature is now dead weight because it was built before we knew who the customer was. **Ask "who is paying for this?" before writing code, not halfway through.**
- **Seed realistic demo data from the start.** Repeatedly resetting and hand-creating test members cost real time. A `seed_demo.py` would have paid for itself in an hour.
- **Decide the deployment target on day one.** The `postgres://` gotcha and the cookie-security setting were both handled early *because* we knew Render was the target. That went well and was worth copying.

Part 12 — What this is actually worth to a gym trainer

Strip away the technology. Here is the business case, which is what your trainer will care about.

The four problems it solves

His problem today	What GymLog does	What it is worth
"Who owes me money?" Lives in a notebook or his head. Some dues quietly never get collected.	Roster shows total pending dues and flags every unpaid member first.	Directly recovers revenue that is currently leaking. **This alone can justify the price.**
"Whose membership is expiring?" He finds out when they have already left.	"Expiring within 7 days" count on the front page, red when expired.	A renewal conversation before expiry converts far better than one after.
"Who is about to quit?" He notices when someone has already been gone a month.	Flags members with no session in 10 days, plus a first-90-days cohort where most churn happens.	Retaining an existing member is far cheaper than acquiring a new one.
"What do I say to this person?" With 30 members on the floor he cannot remember everyone's numbers.	Three factual talking points on each member's page.	This is the product. Members stay for a trainer who *notices* them.

Why the member side matters even though members are not the customer

The member-facing app exists for exactly one reason: **it generates the data the trainer's features run on.** No logs means no stall detection. No attendance means no churn warning.

That is why the member screens are ruthlessly stripped down. Every extra tap is a member who stops logging, and every member who stops logging blinds one of the trainer's instruments. **The member's convenience is the trainer's data quality.**

The honest limitations to tell him up front

- It **does not process payments.** He still collects cash or UPI and records it. That is a deliberate choice, not a gap — taking payments brings legal obligations.
- It **needs him to tap attendance.** Thirty seconds a day. If he does not, the retention features go dark.
- It supports **one trainer.** If he hires a second coach, it needs work.
- On the free tier it is **slow on the first visit** after a quiet period, and the database is temporary.

How to demo it to him. Do not start with the technology — he does not care. Open the trainer dashboard and say: *"This is every member. The red ones owe you money or are about to disappear. This one hasn't been in for 12 days. This one's membership ends Friday."* Then open a member's page and read the talking points aloud. **Show him the answer to a question he already asks himself every morning.**

Part 13 — Decisions you should have questioned

You asked how to work with me better. This is the most direct answer I can give: here are the points in this project where I made a judgement call and you accepted it without pushing back. **Every one of them was a legitimate place to say "why?"**

What I decided	What you should have asked	Was I right?
FastAPI over Django	"Django has a built-in admin panel and user system. Wouldn't that have been less work?"	**Genuinely arguable.** Django's admin would have given the trainer a data-editing interface for free. I would defend FastAPI, but you should have made me justify it.
Stall = no improvement over the last 3 sessions	"Why 3? Why not 4, or a month?"	**It is a guess.** A defensible one, but nobody validated it against how real lifters progress. Ask your trainer whether 3 sessions matches his instinct.
Inactive = 10 days	"Where does 10 come from?"	**Also a guess.** For someone training twice a week, 10 days is about two missed sessions. Reasonable, unvalidated.
Attendance *and* logs count as presence	"The brief said replace logs with attendance. Why are you deviating?"	**I was right, and I flagged it** — a strict swap would have flagged every member on day one. But you took my word for it rather than making me prove it.
Weekly streaks, not daily	"Why not daily, like every other fitness app?"	**Right, and worth understanding:** nobody trains 7 days a week, so a daily streak resets constantly and reads as failure.
YouTube search links instead of specific videos	"Isn't a real video better than a search page?"	**Right for the honest reason:** I cannot verify 48 specific video URLs are live or show correct form. A wrong video for a beginner is worse than a search.
Rupees, hardcoded	"What if he wants a different currency?"	**Fine for now, lazy long-term.** It is a display string in several templates rather than a setting.
Deleting the `/danger/reset` endpoint	"You just built that for me. Why remove it?"	**Right** — a public URL that wipes a database holding payment records is a serious liability. But you should have made me explain the trade-off rather than accepting it.

The pattern to notice

Look at the middle column. Almost every good question is one of four shapes:

1. **"Why this number?"** — whenever you see a specific figure (3 sessions, 10 days, 90 days, 7 days), someone chose it. Ask whether it was measured or guessed. In this project, **guessed**, and that is worth knowing.

2. **"What did you not choose, and why?"** — every choice has alternatives. If I cannot name what I rejected and why, I have not really made a decision.
3. **"What breaks if this is wrong?"** — separates decisions that matter from decisions that do not. Wrong stall threshold: a slightly noisy flag. Wrong permission check: a data breach.
4. **"Show me it working."** — the strongest one. Not "did you test it" but "show me the output." You did this instinctively when you reported the email bug, and **you were right and I was initially looking in the wrong place.**

Part 14 — Your checklist for the next project

Before any code is written

- **Who pays for this, and what do they do every day?** We answered this halfway through and it changed everything. Answer it first.
- **What is the one thing it must do?** If it did only that, would it still be worth using?
- **Where will it run, and what does that platform expect?** Deciding Render early is why the `postgres://` gotcha never bit us.
- **What data will it hold, and how bad is a leak?** Names and payments → security is a phase, not an afterthought.

While building

- **"Where is this rule enforced?"** If the answer is "the button is hidden", it is not enforced.
- **"Is this a real test or are you telling me it works?"** Ask for the output.
- **"What did this break?"** After every change. This is what regression tests answer.
- **"Which version is actually running?"** Your email bug was a stale server, not bad code. Always check what is deployed.
- **"Is that number measured or guessed?"**
- **"What is the simplest thing that works?"** Then ask whether the complicated thing is actually needed.

Before showing anyone

- Reset to clean data and walk the *whole* flow yourself, start to finish.
- Try to break the permissions: log in as the low-privilege user and poke at URLs.
- Check it on the device it will actually be used on. This app was tested at 375px because gym floors mean phones.
- Warm up the free-tier server a few minutes beforehand.

Red flags in my answers — push back when you see these

If I say...	Ask...
"It should work"	"Did you run it? Show me the output."
"This is best practice"	"Best for what? What is the trade-off here?"
"I've added tests"	"How many passed? Did any fail first? What did you change?"
"That's a minor issue"	"Minor for whom? What happens if it goes wrong in front of a customer?"
"I fixed it"	"What was the actual cause? Could it happen anywhere else in the app?"
A confident number with no source	"Did you measure that or estimate it?"

The single most useful habit: when I explain a decision, ask me to name the option I *rejected* and why. A decision without a discarded alternative is not a decision — it is a default. You will find out very quickly whether I actually thought about it.

Glossary

Term	Meaning
API	A way for programs to talk to each other instead of a human clicking. Our `/api/*` routes return raw data rather than a page.
Assertion	A statement in a test that must be true, e.g. "the page contains 102".
Authentication	Proving *who you are* (logging in).
Authorisation	Deciding *what you are allowed to do* once known. The two are constantly confused; the second is where breaches happen.
Backfill	Filling in a value for rows that existed before a column was added — e.g. setting old routine rows to `assigned_by = member`.
bcrypt	A deliberately slow password-scrambling algorithm. Slowness is the security feature.
Cold start	The delay when a sleeping free-tier server wakes up.
Commit	A saved snapshot of the code with a message explaining why.
Cookie	A small piece of data the browser stores and sends back with each request. Ours holds the login token.
CRUD	Create, Read, Update, Delete — the four basic data operations. Our `crud.py` holds that logic.
Dependency	Code written by someone else that your app relies on. Every one is a maintenance and security liability, which is why we kept the list short.
Deploy	Putting new code onto the live server.
Docker	Packaging the app with its exact environment so it runs identically anywhere.
Endpoint / route	One URL the app responds to, e.g. `/dashboard`.
Environment variable	A setting passed in from outside the code — how secrets stay out of the repository.
Framework	A pre-built skeleton for an app. FastAPI is ours.
Hash	A one-way scramble. You can make it, you cannot reverse it.
HTTP status codes	200 = fine. 303 = go here instead. **403 = you are not allowed. 404 = not found.** 500 = the server crashed.
httponly	A cookie flag meaning JavaScript cannot read it — protects the login token.
Idempotent	Doing it twice has the same effect as doing it once. Tapping attendance twice makes one record, not two.
IDOR	Insecure Direct Object Reference — changing an id in a URL to read someone else's data. The main attack we defend against.
Jinja2	The template system that fills placeholders in HTML with real data.
JWT	JSON Web Token — a signed proof of identity the browser carries.
Migration	A change to the shape of the database on an existing system with real data in it.

Term	Meaning
N+1 problem	Running one query, then one more for every result. 1 + 20 queries instead of 2. A classic slow-page cause.
ORM	Object Relational Mapper — lets you use database rows as normal objects. SQLAlchemy is ours.
PostgreSQL	A full database server. Our live database.
Regression	Breaking something that used to work. Regression tests catch it.
Repository (repo)	The project folder tracked by Git, stored on GitHub.
Rolling average	Smoothing noisy data by averaging over a window. Used so water-weight swings do not look like failure.
Salt	Random data mixed into a password before hashing, so identical passwords produce different hashes.
Schema	The structure of the database — tables and columns.
Seeding	Loading starter data automatically, e.g. our 48 exercises.
Server-rendered	The server builds the finished HTML. The opposite of a JavaScript app building it in the browser.
SQL injection	Attacking a database by typing SQL into a form. The ORM prevents it.
SQLite	A database that is just a single file. Our local one.
SVG	Scalable Vector Graphics — shapes described in text. Our charts.
Token	A string proving you are logged in.
UTF-8	The text encoding that supports every character — the arrow and rupee signs.

Closing

You built a real, deployed, security-reviewed web application with a database, two role-based interfaces, automated progress analysis and a live public URL. That is not a toy project.

But the part that matters most for what you asked is this: **you noticed you were nodding along, and you said so.** That is the skill. Tools like me will confidently produce plausible work all day; the value you add is knowing which questions force that work to justify itself.

If you remember three things from this document:

1. If a rule is not enforced on the server, it does not exist.
2. Most failing tests mean the test is wrong — but you must check every time, because the one exception is the bug nobody would have found by clicking.
3. Ask what I *rejected* and why. A decision without a discarded alternative is just a default.

GymLog — project handbook. Covers commits `dfc7cc6` through `ef6d0f0`, Phases 0–6, on branch `main`. Live at gymlog-jtd8.onrender.com.